

Zynq SoC 기반 Plasma Etch OES Virtual Metrology INT8 CNN NPU

프로젝트 설계 검토 및 진행 상세 요약

대화 기록 + 실행 로그 + 업로드 산출물 기반 기술 리뷰

2026-08-16

문서 목적

지금까지의 목적 정의, 데이터 구축, AI model feasibility, INT8/fixed-point 검증, 현재 NPU 설계 시작점까지를 하나의 설계 문서로 재구성한다. 또한 현재 진행 방향이 논리적으로 타당한지, 무엇이 이미 검증됐고 무엇이 아직 미검증인지 구분한다.

범위: 비실시간 오프라인 시연 / Zybo Z7-20 / PS-PL 분할 / Tiny CNN residual regression / INT8 NPU

문서 구성

- 1. Executive Summary - 프로젝트가 무엇이고 현재 어디까지 왔는가
- 2. 프로젝트 목적과 산업적 문제 정의
- 3. 최종 시연 시나리오와 PS/PL 시스템 구성
- 4. 데이터셋과 전처리 설계
- 5. Feasibility 검증 전략과 데이터 누수 방지
- 6. 실험 단계별 코드/결과/판단 기록
- 7. 최종 FP32/INT8 모델 및 수치 규격
- 8. Fixed-point Golden Model 과 RTL 기준
- 9. NPU 마이크로아키텍처 시작점
- 10. 설계 타당성 감사(Design Audit)
- 11. 현재 진행률, 남은 위험, 다음 설계 순서
- 부록 A. 실행 파일/산출물 인덱스
- 부록 B. 핵심 코드 요약
- 부록 C. 주요 결과 로그 요약

중요한 읽기 기준

LOLO/멀티시드/INT8 성능은 일반화 성능 검증용이다. 반면 full 75-wafer 로 다시 학습한 deployment model 의 0.087208 um MAE 는 training-set sanity check 이므로 최종 성능 지표로 사용하면 안 된다.

1. Executive Summary

본 프로젝트는 BOSCH plasma etch 장비에서 수집된 OES(Optical Emission Spectroscopy) 데이터를 이용하여 후공정 측정 전에 Mean Si Etch Depth 를 추정하는 Virtual Metrology 시스템을 Zynq SoC 에서 구현하는 것을 목표로 한다. 핵심은 OES 를 단순 저장/분석하는 것이 아니라, PS 에서 전처리하고 PL 의 INT8 CNN NPU 가 wafer-order 기반 baseline 의 residual 을 보정하도록 구성하는 것이다.

Planned Zynq SoC Virtual Metrology Demo Architecture

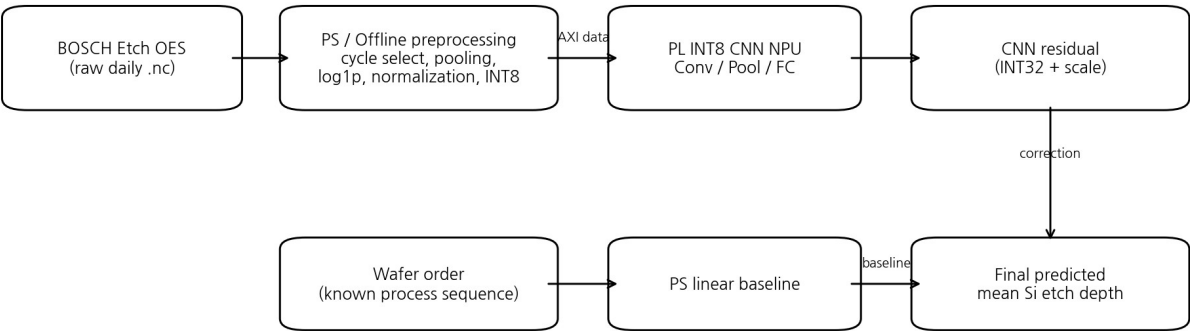


그림 1. 현재 합의된 최종 시스템/시연 구조. PS 가 공정 순서 baseline 과 전처리를 담당하고, PL NPU 가 OES residual 을 추론한다.

현재 결론

데이터/AI feasibility 는 통과했다. 75 개 wafer 전체 데이터 구축, Lot-wise 검증, OES 정보 유효성, residual CNN, multi-seed 안정성, INT8 PTQ, fixed M+shift requantization 까지 완료됐다. 현재는 모델 가능성을 검증하는 단계가 끝나고, BRAM layout / dataflow / 8-MAC / controller 를 정하는 NPU microarchitecture 설계 시작점이다.

영역	현재 상태	핵심 근거
데이터 구축	완료	75/75 wafer PASS, X=(75,100,128,1), 8 lots
OES 예측 정보	검증 완료	OES Ridge MAE 0.2319 vs mean baseline 0.2984
Residual formulation	검증 완료	Wafer+OES Ridge MAE 0.0988, 7/8 lots 개선
Tiny CNN	검증 완료	Residual CNN single run 0.1034, multi-seed mean 0.1163
안정성	검증 완료	5 seeds, std 0.0051, worst 0.1247 < 0.1541
INT8	검증 완료	INT8 MAE 0.1232, FP32 대비 -0.0015 um
Fixed-point	검증 완료	최종 output reference 와 fixed 가 0.0 um 차이
RTL NPU	미구현	개념 단계: shared 8-MAC + ping-pong BRAM 권장
AXI/Zybo Demo	미구현	register map, RTL 통합, 보드 latency/resource 측정 필요

현재 진행률을 두 가지 관점으로 보면

- 기술 위험 제거 기준: 약 70~80% 수준. 가장 큰 초기 위험이었던 “75 wafer 로 예측이 가능한가 / INT8 로 유지되는가”는 해소됐다.
- 실제 구현 완성도 기준: 약 45~50% 수준. RTL, AXI, PS/PL, 보드 실측, 최종 데모가 아직 남아 있다.

Project Position at the End of This Conversation

Data & sync	LOLO / baselines	Residual CNN	Multi-seed	INT8 PTQ	Fixed golden	Memory / dataflow	RTL layers	AXI + Zybo	Demo
DONE	DONE	DONE	DONE	DONE	DONE	NEXT	TODO	TODO	TODO

Feasibility risk has been largely retired; implementation risk now dominates.

그림 2. 이 대화 종료 시점의 프로젝트 위치.

2. 프로젝트 목적과 산업적 문제 정의

2.1 문제 정의

Plasma etch 공정에서는 최종 wafer 의 etch depth 를 확인하기 위해 ex-situ 계측이 필요하다. 본 프로젝트는 장비 내부에서 공정 중 이미 취득되는 OES 를 활용해 etch 결과를 추정하고, 실제 계측 전에 공정 결과를 빠르게 예측하는 Virtual Metrology(VM) 형태를 구현한다.

프로젝트의 실제 목적

“FPGA/CNN 을 보여주기 위한 데모”가 아니라, BOSCH plasma etch 중 발생하는 spectral-temporal OES 정보를 이용해 최종 Mean Si Etch Depth 를 추정하는 공정/장비용 VM accelerator 를 구현하는 것.

2.2 왜 단순 OES-only regression 이 아니라 residual 구조인가

데이터 분석 결과 wafer order 만으로도 chamber-state drift 를 상당 부분 설명할 수 있었다. Wafer-order linear baseline 의 LOLO MAE 는 0.1541 um 으로, mean baseline 0.2984 um 보다 훨씬 좋았다. 따라서 최종 문제를 “OES 가 40.xx um 절대값 전체를 직접 예측”하도록 두는 대신, PS 의 wafer-order baseline 이 설명하지 못한 residual 을 CNN NPU 가 보정하는 구조로 재정의했다.

```
baseline_um = baseline_coef * wafer_order + baseline_intercept
residual_um = cnn_int32_output * output_um_scale + residual_mean
final_etch_depth_um = baseline_um + residual_um
```

이 재정의가 핵심 전환점이었다. Direct Tiny CNN 은 평균 예측으로 붕괴했지만, residual CNN 은 OES 가 가진 추가 공정 상태 정보를 안정적으로 추출했다.

2.3 프로젝트가 주장해야 하는 범위

- 주장 가능: 동일한 데이터셋에서 Lot-wise held-out 평가를 했을 때 OES residual CNN 이 wafer-order baseline 을 개선했다.
- 주장 가능: INT8 PTQ 가 FP32 성능을 사실상 유지했다.
- 아직 주장 불가: 새로운 fab/장비/다른 recipe 까지 일반화한다.
- 아직 주장 불가: FPGA 가 ARM 보다 실제로 몇 배 빠르다. 보드 실측 전에는 latency/speedup 수치를 확정하면 안 된다.
- 주의: Gas5 $\approx$ SF6, Gas4 $\approx$ C4F8 매핑은 cycle timing 을 근거로 한 강한 추정이며 feature name 자체가 chemistry 를 직접 표기한 것은 아니다.

3. 최종 시연 시나리오와 PS/PL 시스템 구성

3.1 보드와 구현 원칙

항목	현재 설계
Target board	Zybo Z7-20 (Zynq-7020)
실시간 요구	없음. 비실시간/offline VM demo
PS 역할	OES 전처리, wafer-order baseline, AXI control/data transfer, final dequantization/합산
PL 역할	INT8 CNN NPU inference
Control interface	AXI4-Lite 권장
Tensor/weight transfer	초기 구현은 AXI BRAM Controller 또는 BRAM window 방식 권장; streaming/DMA 는 후순위
정수 규격	INT8 activation/weight, INT32 bias/accumulator, fixed requantization

3.2 계획된 데모 흐름

1. PC 또는 PS 가 한 wafer 의 전처리된 100x128 INT8 OES tensor 와 wafer order 를 준비한다.
2. PS 가 wafer-order linear baseline 을 계산한다.
3. PS 가 tensor/weights/control 을 PL NPU 로 전달하고 start 를 건다.
4. PL NPU 가 Conv1 -> Pool1 -> Conv2 -> Pool2 -> AvgPool -> FC1 -> FC2 를 실행한다.
5. FC2 의 INT32 accumulator 를 PS 가 읽고 output_um_scale 을 적용해 residual_um 으로 복원한다.
6. PS 가 baseline + residual 을 합산해 Mean Si Etch Depth 를 출력한다.
7. 데모 화면에는 measured value, predicted value, absolute error, ARM/PL latency, resource/timing 결과를 함께 표시하는 것이 적합하다.

3.3 시연에서 반드시 구분해야 할 것

- 학습은 PC 에서 수행한다. FPGA 는 inference 만 수행한다.
- log1p, normalization, input quantization 은 초기 demo 에서는 PS/소프트웨어 전처리로 두는 것이 구현 위험을 낮춘다.
- NPU 의 핵심 성과는 “CNN inference 를 정수 연산으로 정확히 재현하고 실제 하드웨어로 실행”하는 것이다.
- VM prediction accuracy 는 LOLO 평가 수치를 사용하고, deployment full-data fit 수치를 성능처럼 제시하지 않는다.

4. 데이터셋과 전처리 설계

4.1 데이터 구성

데이터	구조/역할
Daily OES .nc	wafer 별 OES time x wavelength data. 각 wafer 에서 3648 spectral channels
Process_data.nc	wafer 별 31 process variables, process-relative time
Dictionary_OES.nc	lossless encoded OES value decoder
Dictionary_process.nc	encoded process value decoder
Si_Oxide_etch_9_points.csv	wafer 당 9 개 측정점의 si_etch 등 post-process measurement
Lot metadata	date/lot/wafer/experiment_key 를 연결해 leakage-safe LOLO split 에 사용

4.2 최종 사용 샘플

Lot	Date	Usable wafers
1	2024-07-02	10
2	2024-07-05	10
3	2024-07-09	10
4	2024-07-11	10
5	2024-07-19	10
6	2024-08-01	10
7	2024-08-05	5
9	2024-08-21	10
합계		75

직접 업로드된 full_dataset_raw.npz 를 재검사한 결과 $X=(75, 100, 128, 1)$, $y=(75,)$, target min=39.5544, max=40.8170, mean=40.2331, std(ddof=0)=0.3440 um 이다.

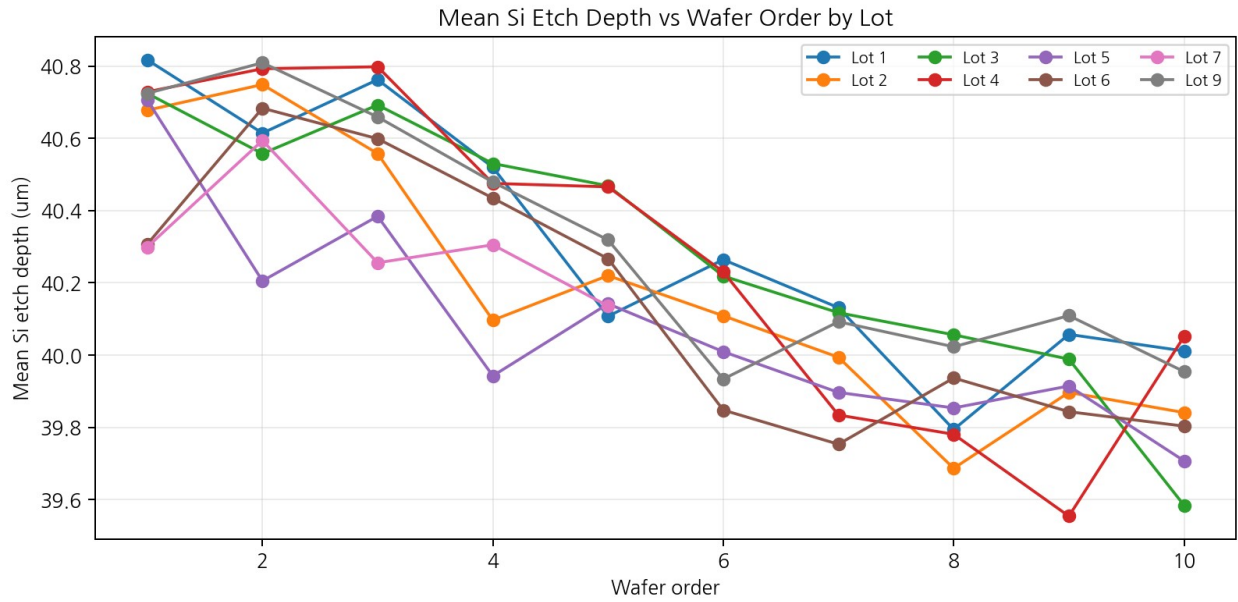


그림 3. Lot 별 wafer 순서에 따른 Mean Si Etch Depth. Wafer-order baseline 이 강한 이유를 시각적으로 확인할 수 있다.

4.3 OES - Process 동기화

OES timestamps와 process timestamps의 표현 방식이 달랐기 때문에 단순 index alignment를 사용하지 않았다. 495~505 nm OES band 평균과 Gas5 time-series의 cross-correlation을 사용해 wafer 별 offset을 계산했다. 초기 ± 10 s 탐색은 약 6 s BOSCH periodicity 때문에 alias가 생겨 ± 3 s로 제한했다.

QC 항목	75-wafer 결과
PASS	75 / 75
OES-process offset	0.10 ~ 0.25 s, mean 0.184 s
Correlation	-0.777 ~ -0.679, mean -0.757
Input intensity min (wafer-wise)	10.42 ~ 14.92
Input intensity max (wafer-wise)	3100.5 ~ 3937.8

4.4 BOSCH cycle / Gas5 선택

- Process Gas5 threshold > 300을 이용해 high phase를 잡고 rising edge로 cycle을 분리했다.
- 각 wafer에서 100개의 main Gas5 phase를 선택했다.
- 2024-08-05 Wafer_03에서는 초기 interval이 15 s인 startup anomaly가 있었지만 steady-state cycle은 정상이라 wafer를 폐기하지 않았다.
- 첫 CNN input은 Gas5 etch phase OES만 사용했다. final Gas4 passivation phase가 모든 cycle에 완전하게 존재하지 않는 startup/ending 구조를 억지로 패딩하지 않았다.

4.5 최종 CNN 입력 표현

한 wafer를 $100 \times 128 \times 1$ tensor로 변환했다. 세로축은 100개의 Gas5 etch phases, 가로축은 3648 wavelength channel을 128 bin으로 max-pooling한 spectral axis이다. 각 Gas5 phase에서 해당 OES sample들의 spectrum을 평균한 뒤 wavelength 방향 max-pool을 수행했다. max-pool은 narrow emission peak를 보존하려는 선택이었다.

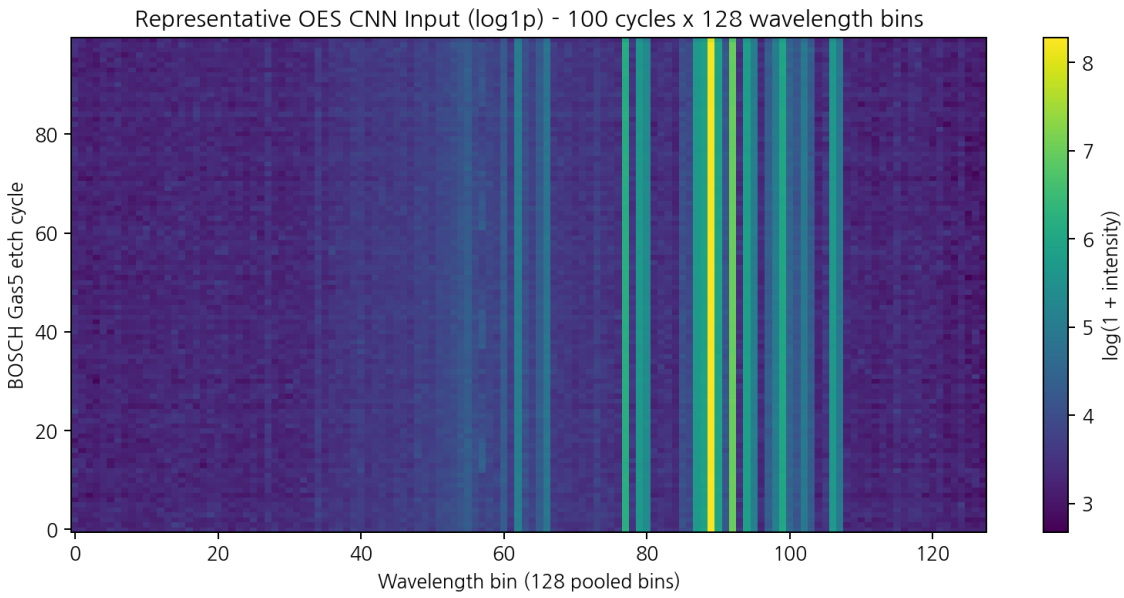


그림 4. 실제 *full_dataset_raw.npz*의 첫 sample을 *log1p*로 시각화한 100x128 OES map.

Leakage 방지 원칙

100x128 tensor 생성 자체는 고정된 변환이지만, *log1p* 이후 mean/std normalization 과 target normalization 은 각 LOLO fold 의 training lots 에서만 계산했다. Test lot 통계는 preprocessing fit 에 사용하지 않았다.

5. Feasibility 검증 전략과 평가 방법

5.1 왜 random split 을 쓰지 않았는가

같은 lot 의 연속 wafer 는 chamber state drift 와 recipe history 를 공유한다. random wafer split 을 사용하면 train/test 에 인접 wafer 가 섞이면서 과도하게 높은 성능이 나올 수 있다. 따라서 8 개 lot 를 기준으로 Leave-One-Lot-Out(LOLO)을 사용했다.

```
for test_lot in unique_lots:
    train_mask = lots != test_lot
    test_mask = lots == test_lot
    # fit preprocessing/model only with train lots
```

5.2 평가 지표

지표	용도
MAE	물리 단위 um 에서 평균 오차를 직접 해석. 주요 판단 지표.
RMSE	큰 오차에 더 민감. fold collapse 여부 확인.
R2	분산 설명력 확인. target range 가 좁아 MAE 와 함께 해석.
Fold 별 MAE	특정 lot 가 overall score 를 지배하는지 확인.
Multi-seed mean/std	작은 데이터에서 초기화 민감도/학습 안정성 확인.

5.3 GO/NO-GO gate 가 실제로 어떻게 진행됐는가

Gate	질문	결과
G1	75 wafer 를 일관된 tensor/target 으로 만들 수 있는가?	PASS - 75/75
G2	OES 자체에 target 정보가 있는가?	PASS - OES Ridge 0.2319 < mean 0.2984
G3	Wafer-order 외 추가 정보가 있는가?	PASS - residual Ridge 0.0988 < 0.1541
G4	작은 CNN 도 residual 을 학습하는가?	PASS - 0.1034 single run
G5	seed 가 바뀌어도 유지되는가?	STRONG PASS - mean 0.1163, std 0.0051
G6	INT8 로 바뀌도 유지되는가?	STRONG PASS - 0.1232
G7	실수 multiplier 없이 fixed requant 가능한가?	PASS - final output 0 difference

6. 실험 단계별 코드 / 결과 / 판단 기록

아래는 실제 대화에서 작성/실행한 스크립트 흐름을 시간 순서대로 재구성한 것이다.

6.1 데이터 구조 확인 및 2024-08-05 pilot

스크립트/작업	핵심 내용	결과
check_process.py	Wafer_01 process group 의 31 feature, Gas4/Gas5 pulse, time axis 확인	Gas5 101 rises, main 100 phases 선정 가능
crop_bosch_oes.py	OES time range 및 BOSCH active 구간 확인	약 598.7 s crop 후보 확인
build_cnn_input.py	Gas5 phase mean spectrum -> 128 max-pool	(100,128,1), zero/missing cycle 없음
build_batch_inputs.py	Aug-05 Wafer_01~05 batch 처리	5/5 PASS
build_dataset_2024_08_05.py	5 개 input 과 9-point mean si_etch 연결	X=(5,100,128,1), y=(5,)
inspect_full_dataset.py	전체 target/lot/date/required OES 목록 확인	75 usable wafers, 8 lots

6.2 full_dataset_raw 구축

- usable_experiments.csv 기준으로 labeled wafer 만 선택
- Process_data.nc 에서 Gas5 추출 및 100-cycle 후보 선택
- OES 495~505 nm vs Gas5 cross-correlation 으로 wafer 별 offset 추정
- Gas5 high phase OES 만 decode 하고 phase 평균 spectrum 생성
- 3648 wavelength -> 128 max-pool
- 9 개의 si_etch 를 wafer 별 평균해 target 연결
- NaN/Inf, cycle, group, correlation 등 QC 기록
- full_dataset_raw.npz + full_dataset_qc.csv 저장

```
Generated samples : 75 / 75
Failed samples   : 0
X shape          : (75, 100, 128, 1)
y shape          : (75,)
Lots             : [1 2 3 4 5 6 7 9]
Target min/max   : 39.5544 / 40.817
Target mean/std  : 40.23312 / 0.34401816
```

6.3 Baseline: mean vs wafer-order

모델	MAE (um)	RMSE (um)	R2
Train mean baseline	0.2984	0.3474	-0.0197
Wafer-order linear baseline	0.1541	0.1885	0.6998

해석

wafer order 자체가 chamber drift/lot progression 의 강한 proxy 다. 따라서 OES 모델이 mean baseline 만 이기는 것으로는 충분하지 않고, wafer-order baseline 에 추가적인 정보를 제공해야 프로젝트 의미가 강해진다.

6.4 첫 Direct Tiny CNN 실패

```
Input 100x128x1
-> Conv3x3(1->4) -> ReLU -> MaxPool
-> Conv3x3(4->8) -> ReLU -> MaxPool
-> Global Average Pool -> FC(8->1)
```

항목	결과
MAE	0.2989 um
RMSE	0.3488 um
R2	-0.0277
Prediction std	약 0.0285 um
Target std	약 0.3440 um

예측 분산이 target 에 비해 거의 0 으로 줄어 평균 근처에 collapse 했다. 여기서 모델이 실패했다고 바로 OES 를 버리지 않고, “OES 신호가 없는지 vs CNN 구조가 못 잡는지”를 분리 진단했다.

6.5 OES Ridge 진단

```
mean_spectrum = np.mean(X_log, axis=1) # 128
slope = cycle-wise linear slope per bin # 128
features = np.concatenate([mean_spectrum, slope], axis=1) # 256
```

모델	MAE	R2	판단
Mean baseline	0.2984	-0.0197	기준
OES Ridge	0.2319	0.2932	OES 에 유효 정보 존재
Wafer-order baseline	0.1541	0.6998	OES-only 보다 강함

6.6 Wafer-order + OES residual Ridge

```
base_train = LinearRegression(wafer_order -> y)
residual_train = y_train - base_train_pred
residual_model = Ridge(OES_features -> residual_train)
final_pred = base_test_pred + predicted_residual
```

Overall	결과
MAE	0.0988 um
RMSE	0.1261 um
R2	0.8656
Wafer baseline 대비 개선	35.9%

Lot	Wafer baseline MAE	+OES residual MAE	변화
1	0.1654	0.0806	51.3% 개선
2	0.1111	0.1531	37.8% 악화
3	0.1228	0.1102	10.3% 개선
4	0.2072	0.1090	47.4% 개선
5	0.1696	0.1193	29.7% 개선
6	0.1402	0.0373	73.4% 개선
7	0.1751	0.1119	36.1% 개선
9	0.1520	0.0759	50.0% 개선

결론

8 개 lot 중 7 개가 개선됐다. OES 가 단순 wafer-order drift 의 복제 신호만 갖고 있는 것이 아니라 baseline residual 을 설명하는 추가 정보를 갖는다는 강한 근거가 확보됐다.

6.7 Position-preserving Residual Tiny CNN

```
100x128x1
-> Conv3x3 1->4 -> ReLU -> MaxPool      = 50x64x4
-> Conv3x3 4->8 -> ReLU -> MaxPool      = 25x32x8
-> AvgPool(5x4)                               = 5x8x8
-> Flatten 320 -> FC 16 -> ReLU -> FC 1
-> residual
```

항목	결과
Trainable parameters	5,489
Wafer baseline MAE	0.1541 um
Residual Tiny CNN MAE	0.1034 um
RMSE	0.1358 um
R2	0.8441
Baseline 대비 개선	32.9%
Lot 개선	8/8 in this run

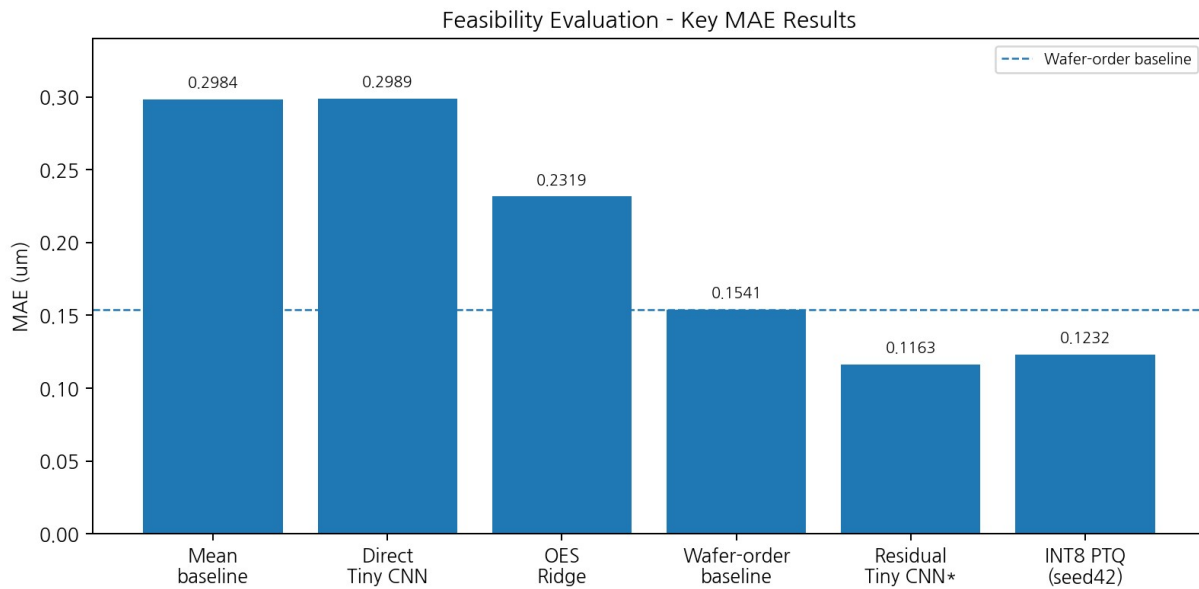


그림 5. 주요 feasibility 실험의 MAE 진행. Direct CNN 실패 후 residual formulation 이 성능 전환점이었다.

6.8 Multi-seed 안정성

Seed	MAE	RMSE	R2	Improved lots
7	0.1171	0.1502	0.8094	7/8
21	0.1131	0.1399	0.8347	7/8

42	0.1247	0.1531	0.8020	6/8
84	0.1144	0.1509	0.8076	8/8
123	0.1121	0.1416	0.8305	8/8

Stability statistic	값
Mean MAE	0.1163 μm
MAE std	0.0051 μm
Best MAE	0.1121 μm
Worst MAE	0.1247 μm
Mean R2	0.8169
R2 std	0.0147

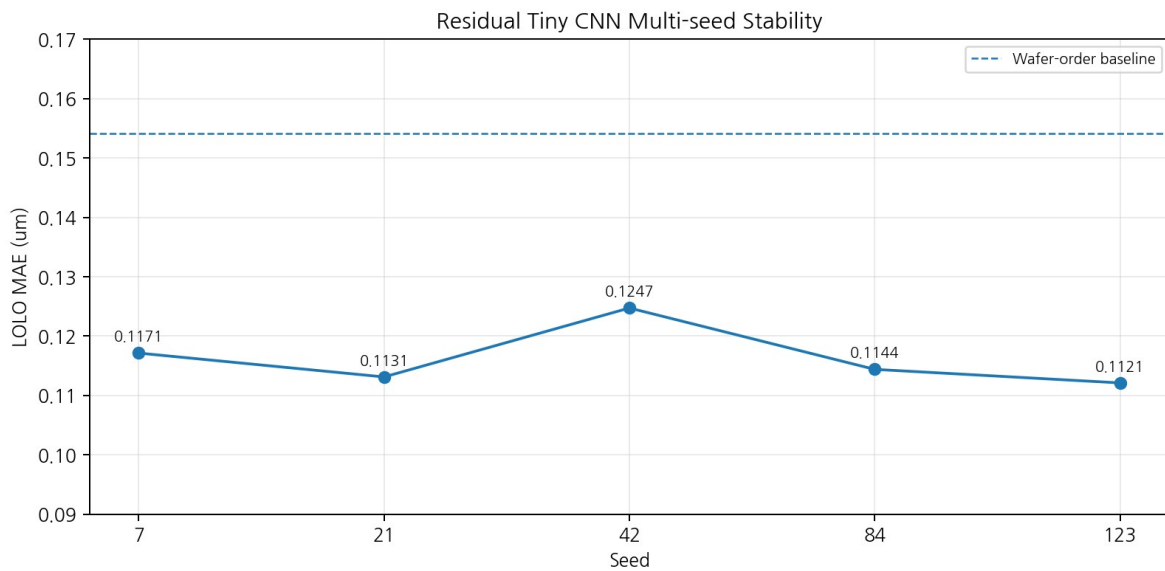


그림 6. 5 개 seed 에서 모두 wafer-order baseline(0.1541 μm) 아래로 유지.

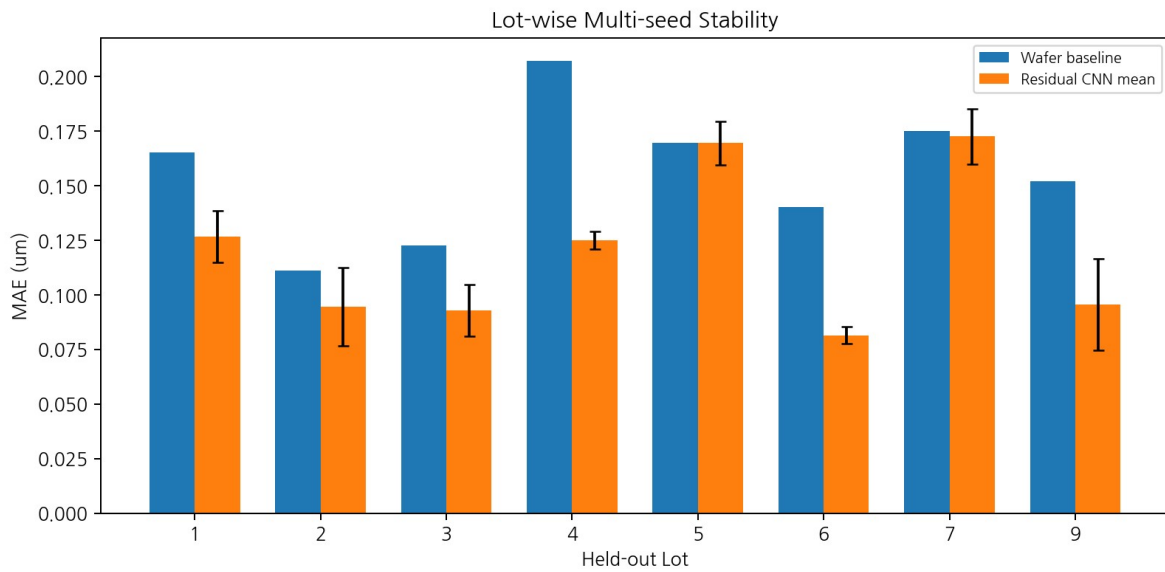


그림 7. Lot-wise multi-seed 평균. Lot 5 와 Lot 7 이 상대적으로 약한 구간이다.

6.9 INT8 PTQ feasibility

모델	MAE	RMSE	R2
Wafer baseline	0.1541	0.1885	0.6998
FP32 (seed42 conservative setup)	0.1247	0.1531	0.8020
INT8 PTQ	0.1232	0.1517	0.8056

INT8 판정

INT8 MAE degradation = -0.0015 um. 이는 “INT8 가 더 우수하다”는 의미보다 quantization 으로 인한 성능 손실이 사실상 없었다는 의미다. INT8 feasibility 는 STRONG PASS 로 판단했다.

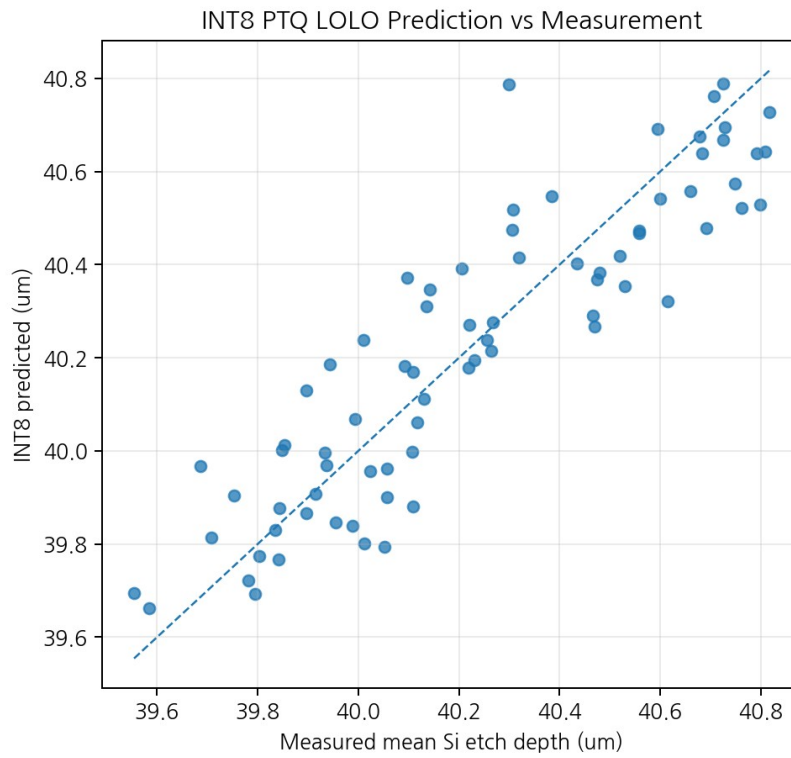


그림 8. INT8 LOLO prediction vs measured target.

6.10 Deployment model freeze

배포 파라미터	값
Wafer baseline coefficient	-0.10232342
Wafer baseline intercept	40.77884674
Residual mean	0.00000041
Residual std	0.17961393
Input log1p mean	3.79108977
Input log1p std	0.83221191
s_input	0.0424592026
s_a1	0.0719836528
s_a2	0.0534594303
s_a3	0.0253116765
s_w1	0.0046707969
s_w2	0.0061882744
s_w3	0.0063263830
s_w4	0.0032723356
Output physical scale	0.000014877117 um/count

6.11 Fixed M + Shift Golden

Layer	Real multiplier	M	Shift	M width
Conv1	0.002755046515	46,222	24	16 bit
Conv2	0.008332572775	139,797	24	18 bit
FC1	0.013361613240	224,171	24	18 bit

Layer compare	Mismatch rate	Max diff
Conv1 ReLU	0.000547%	1 LSB
Pool1	0.000729%	1
Conv2 ReLU	0.001615%	1
Pool2	0.002292%	1
AvgPool	0.004167%	1
FC1 ReLU	0.000000%	0
FC2 accumulator	0.000000%	0
Final physical output	0.000000%	0.0000000000 um

Golden Model 해석

중간 tensor에서 극소수 1-LSB mismatch가 발생했지만 FC1 이후 완전히 소멸해 75개 sample의 final output은 reference PTQ와 fixed model이 완전히 동일했다. 이제 RTL은 fixed model을 기준으로 검증해야 한다.

7. 최종 모델 및 수치 규격

7.1 Frozen network topology

Stage	Input	Operation	Output
Input	1x100x128	INT8 tensor	1x100x128
Conv1	1x100x128	3x3, Cin=1, Cout=4, pad=1	4x100x128
ReLU + MaxPool	4x100x128	ReLU, 2x2 stride2	4x50x64
Conv2	4x50x64	3x3, Cin=4, Cout=8, pad=1	8x50x64
ReLU + MaxPool	8x50x64	ReLU, 2x2 stride2	8x25x32
AvgPool	8x25x32	5x4 stride 5x4	8x5x8
Flatten		CHW flatten	320
FC1	320	320->16	16
ReLU	16	clamp negative to 0	16
FC2	16	16->1	INT32 accumulator

7.2 Parameter size / MAC count

Layer	Weights	MACs
Conv1	36 B	460,800
Conv2	288 B	921,600
FC1	5,120 B	5,120
FC2	16 B	16
합계	5,460 B	1,387,536

8-MAC shared engine 을 가정하면 MAC-only ideal lower bound 는 $1,387,536 / 8 = 173,442$ cycles 이다. 100 MHz 가정 시 약 1.73 ms 이지만, 이는 BRAM read/write, padding, requant, pooling, controller overhead 를 제외한 하한값이다. 실제 latency 는 RTL/board 측정 전에는 확정하지 않는다.

7.3 Activation memory footprint

Tensor	Elements	INT8 bytes
Input 1x100x128	12,800	12.8 KB
Conv1 4x100x128	51,200	51.2 KB
Pool1 4x50x64	12,800	12.8 KB
Conv2 8x50x64	25,600	25.6 KB
Pool2 8x25x32	6,400	6.4 KB
AvgPool 8x5x8	320	320 B
FC1	16	16 B

7.4 Integer arithmetic contract

```
Input/activation/weight : signed INT8, clamp [-127, 127]
Bias                    : signed INT32
MAC accumulator         : signed INT32 contract
ReLU                   : max(q, 0)
Requant                 : round_half_away_from_zero(acc * M / 2^24)
Saturation               : clamp to [-127, 127]
FC2                     : no requant; keep INT32 accumulator
Physical residual       : fc2_acc * 1.487711674e-5 + residual_mean
Final output            : baseline_coef*wafer + baseline_intercept + residual
```

이론적 worst-case accumulator 폭(현재 operand bound 및 frozen bias 를 기준으로 한 상한)은 Conv1 약 19 bit, Conv2 약 21 bit, FC1 약 24 bit, FC2 약 19 bit 면 충분하지만, 상위 인터페이스/검증 단순성을 위해 logical accumulator contract 는 INT32 로 유지하는 것이 안전하다. Requant product 는 M 이 최대 18 bit 이므로 RTL 에서 signed product width 를 충분히 확보해야 한다.

8. Fixed-point Golden Model 과 RTL 기준

8.1 Golden sample 의 역할

rtl_golden_sample0.npz 에는 첫 wafer(2024-07-02_01)의 input_q 부터 각 layer output, FC2 accumulator, final prediction 이 저장돼 있다. 향후 Verilog testbench 에서 layer 별로 이 값과 비교하면 오류 위치를 빠르게 찾을 수 있다.

Field	Shape / value
experiment_key	2024-07-02_01
wafer	1.0
input_q	(1, 100, 128)
conv1_relu	(4, 100, 128)
pool1	(4, 50, 64)
conv2_relu	(8, 50, 64)
pool2	(8, 25, 32)
avgpool	(8, 5, 8)
fc1_relu	(16,)
fc2_acc	[7859]
final_prediction	40.793444783 um

8.2 RTL 전에 반드시 하나 더 명시할 부분

AvgPool divide-by-20 의 정수 규칙

현재 Python golden 은 summed/20.0 에 round-half-away-from-zero 를 적용한다. sum 범위가 작아 float64 에서 정확하지만, RTL bit-exact spec 에는 실수 연산이 없어야 한다. 따라서 양수는 (sum+10)/20 의 integer floor, 음수는 -(((sum)+10)/20)로 정의해 동일한 rounding 을 명시하는 것이 좋다.

```
if (sum >= 0)
    avg = (sum + 10) / 20;
else
    avg = -(((sum) + 10) / 20);
```

8.3 Memory ordering 도 Golden 과 동일하게 고정해야 함

현재 PyTorch/Golden 내부 tensor 는 NCHW 로 계산된다. RTL memory layout 은 CHW contiguous 를 권장한다. 주소식은 channel * H * W + row * W + col 로 단순화할 수 있다.

```
addr(ch, row, col) = ch*(H*W) + row*W + col
Conv1 input (C=1): addr = row*128 + col
Conv1 output:      addr = ch*12800 + row*128 + col
```

9. NPU 마이크로아키텍처 시작점

Recommended Initial NPU Microarchitecture

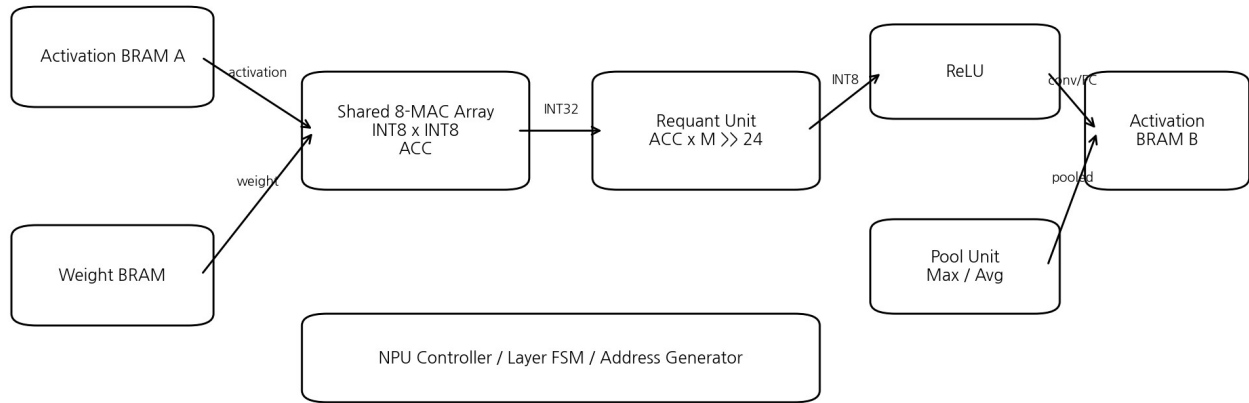


그림 9. 권장 초기 NPU 구조. 하나의 shared MAC engine 을 Conv/FC 가 재사용하고 BRAM ping-pong 으로 layer 를 순차 실행한다.

9.1 왜 shared 8-MAC 인가

- 모델 전체 weight 가 약 5.46 KB 로 작고 연산량은 약 1.39M MAC 이라, layer 별 전용 accelerator 를 만들 필요가 없다.
- 3 인 / 약 2 주 프로젝트에서 streaming line-buffer + fully pipelined multi-engine 구조는 검증 부담이 너무 크다.
- shared MAC 은 Conv/FC 를 하나의 검증된 datapath 로 통합할 수 있어 bit-exact debugging 이 쉽다.
- 후속 최적화가 필요하면 MAC 수나 weight broadcast/data reuse 를 확장하면 된다. 첫 구현은 기능 correctness 우선이 맞다.

9.2 권장 top-level 블록

RTL block	책임
npu_top	AXI/BRAM interface, start/done, layer sequence
npu_controller	layer FSM, loop counter, address generation, ping-pong buffer select
mac_array_8	8 개의 signed INT8 multiply 와 accumulation schedule
requant_unit	signed acc x M, rounding, >>24, saturation
maxpool_unit	2x2 max
avgpool_unit	5x4 sum + exact signed rounded /20
activation_bram_A/B	input/intermediate ping-pong
weight_bram	frozen INT8 weights / INT32 biases

9.3 아직 확정하지 않은 핵심 설계 선택

- 8-MAC 이 한 cycle 에 “8 output channels”를 병렬 계산할지, “8 kernel products”를 병렬 계산할지 세부 schedule 은 아직 미확정.
- BRAM word width(8/32/64 bit), pack format, dual-port 사용 방식이 미확정.
- Padding 을 address generator 에서 zero injection 할지 padded input buffer 를 만들지 미확정.

- AXI BRAM Controller vs custom AXI master/DMA 는 미확정. 첫 구현은 BRAM window 방식이 단순하다.
- Layer descriptor 를 microcode/table 로 일반화할지, 4-layer fixed FSM 으로 하드코딩할지 미확정. 2 주 범위에서는 fixed FSM 이 현실적이다.

9.4 권장 구현 순서

1. Arithmetic spec 문서 동결: signed widths, M/shift, rounding, saturation, AvgPool /20.
2. CHW memory layout 과 각 tensor base address 를 확정한다.
3. Conv1 address generator 를 먼저 설계한다.
4. 1-MAC reference datapath 를 만들고 golden sample 의 Conv1 한 pixel 을 맞춘다.
5. 8-MAC 으로 확장하고 Conv1 전체 tensor 를 bit-exact 비교한다.
6. requant/ReLU/MaxPool 까지 묶어 Pool1 golden 을 맞춘다.
7. Conv2 를 같은 engine 으로 실행한다.
8. AvgPool -> FC1 -> FC2 를 연결한다.
9. npu_controller 로 전체 layer FSM 을 통합한다.
10. rtl_golden_sample0 전체 pass 후 여러 sample testbench 로 확장한다.
11. 그 다음에 AXI4-Lite register map 과 PS/PL transfer 를 붙인다.

10. 설계 타당성 감사(Design Audit)

10.1 잘 설계되고 있는 부분

항목	평가	근거
산업 목적	좋음	OES -> etch depth VM 이라는 실제 공정/장비 문제로 정의됨
데이터 split	매우 좋음	random split 대신 lot-wise holdout 사용
Leakage 통제	좋음	normalization/calibration 을 train lots 로 제한
실패 대응	좋음	Direct CNN 실패 후 데이터 자체와 모델 구조를 분리 진단
문제 재정의	매우 좋음	wafer-order baseline + OES residual 로 물리/데이터 구조를 반영
소형 모델	좋음	5,489 params 로 FPGA 구현 가능성을 염두에 둠
안정성	매우 좋음	multi-seed + lot-wise 결과 확인
Quantization	매우 좋음	INT8/INT32 를 Python 에서 직접 재현
RTL 준비	매우 좋음	M+shift 와 layer golden tensor 까지 확보

10.2 현재 설계에서 남은 리스크

리스크	현재 영향	대응
독립 sample 수 75	높음	LOLO 로 완화했지만 external validation 은 없음. 성능 claim 범위 제한.
Lot 5 / Lot 7 약한 성능	중간	multi-seed 에서 지속 관찰. 최종 보고 시 limitation 으로 제시.
Gas5 chemistry mapping	낮음~중간	timing-based inference 라고 명시. 모델은 Gas5 signal 을 feature 로 사용.
PS preprocessing	중간	log1p/normalization/input quantization 의 embedded 구현과 latency 미측정.
AvgPool exact rounding	낮음	integer /20 rule 을 RTL spec 에 명시하면 해소.
NPU schedule	중간	8-MAC 내부 parallelization 과 BRAM width 결정 필요.
Board resource/timing	미검증	synthesis/implementation 후 LUT/FF/BRAM/DSP/WNS 측정 필요.
ARM vs NPU speedup	미검증	동일 input, 동일 preprocessing boundary 로 latency benchmark 필요.

10.3 가장 중요한 방법론적 주의

최종 프로젝트 설명 문장

“OES 만으로 etch depth 를 완전히 예측하는 CNN”이라고 설명하면 실제 설계와 다르다. 현재 검증된 구조는 “wafer-order baseline

으로 공정 진행 drift 를 모델링하고, OES 기반 INT8 CNN NPU 가 그 residual 을 보정하는 Virtual Metrology SoC”다.

10.4 지금 방향을 바꿔야 하는가?

현재 결과만 보면 주제를 변경하거나 model family 를 다시 탐색할 이유는 없다. Feasibility gate 를 단계적으로 통과했고 INT8/fixed-point 까지 성능이 유지됐다. 이 시점에서 가장 큰 위험은 AI 성능이 아니라 RTL 구현/통합 위험으로 이동했다. 따라서 다음 의사결정은 “더 좋은 모델을 찾을까?”가 아니라 “현재 frozen model 을 가장 단순하고 검증 가능하게 하드웨어로 옮길까?”가 되어야 한다.

11. 현재 진행률, 남은 작업, 다음 마일스톤

Work package	진행률 추정	상태
데이터/OES/target pipeline	100%	완료
AI feasibility / LOLO	100%	완료
Residual CNN / multi-seed	100%	완료
INT8 PTQ / deployment freeze	100%	완료
Fixed-point arithmetic/golden	95%	AvgPool integer rule 문서화만 추가 권장
NPU microarchitecture	15%	개념 구조만 결정
Layer RTL	0%	미착수
Full NPU controller	0%	미착수
AXI / PS-PL	0%	미착수
Board validation / demo	0%	미착수

11.1 바로 다음 3 개의 설계 산출물

1. NPU Arithmetic Specification: signed width, M/shift, rounding, saturation, AvgPool /20, FC2 output scale 를 1 페이지 표로 동결.
2. Tensor/BRAM Memory Map: CHW layout, base address, ping-pong A/B, weight/bias address 를 표로 동결.
3. Conv/FC Cycle Schedule: 8-MAC 이 한 cycle 에 무엇을 소비/생성하는지 예제 pixel 기준으로 작성.

11.2 RTL 단계의 검증 기준

Milestone	Pass condition
Conv1 MAC	한 output pixel accumulator 가 Python golden 과 일치
Conv1+requant	INT8 activation 이 golden 과 bit-exact
Pool1	전체 4x50x64 tensor bit-exact
Conv2/Pool2	각 layer tensor bit-exact
AvgPool/FC1	320 flatten 및 16 activation bit-exact
FC2	INT32 accumulator bit-exact
NPU top	sample0 full pipeline pass
Multi-sample RTL	여러 wafer sample 에서 pass
Synthesis	timing closure + resource report 확보
Board	PS input -> PL NPU -> final prediction end-to-end

11.3 최종 데모에서 보여줘야 할 지표

- Measured Mean Si Etch Depth vs predicted depth
- Absolute prediction error (um)
- LOLO INT8 accuracy summary: MAE 0.1232 um, R2 0.8056
- NPU latency (hardware measured)

- PS/ARM reference latency (same inference scope 기준)
- Speedup, LUT/FF/BRAM/DSP, WNS/TNS
- Python fixed golden vs RTL output bit-exact verification screenshot/log

부록 A. 실행 파일 / 산출물 인덱스

Script	역할
check_process.py	Process feature/time/Gas pulse inspection
crop_bosch_oes.py	OES BOSCH active range inspection
build_cnn_input.py	Single-wafer 100x128x1 input
build_batch_inputs.py	Aug-05 batch preprocessing
build_dataset_2024_08_05.py	5-wafer pilot dataset
inspect_full_dataset.py	75 labeled wafer metadata/target inspection
build_full_dataset.py	75-wafer preprocessing + QC + full_dataset_raw.npz
evaluate_baselines.py	LOLO mean and wafer-order baselines
train_tiny_cnn.py	Direct Tiny CNN - failed diagnostic model
evaluate_oes_ridge.py	OES mean+slope 256-feature Ridge
evaluate_oes_residual.py	Wafer-order + OES residual Ridge
check_residual_consistency.py	Lot-wise residual improvement consistency
train_residual_cnn.py	Position-preserving residual Tiny CNN
check_multiseed_stability.py	5-seed x 8-lot stability
evaluate_int8_ptq.py	INT8 x INT8 -> INT32 PTQ feasibility
train_export_deployment_model.py	Full-data deployment model freeze/export
build_fixed_golden.py	Integer M+shift requant + RTL golden export

Artifact	내용
full_dataset_raw.npz	X/y/lot/wafer/date/experiment_key; 75 samples
full_dataset_qc.csv	75/75 PASS, offset/correlation/input range/target QC
tiny_cnn_lolo_result.npz	Direct CNN LOLO prediction
residual_tiny_cnn_result.npz	Residual CNN prediction
multiseed_summary.csv	seed-wise MAE/RMSE/R2
multiseed_lot_results.csv	seed x lot fold results
multiseed_predictions.npz	5-seed prediction arrays
int8_ptq_result.npz	baseline/FP32/INT8 LOLO predictions
deployment_fp32_model.pt	frozen FP32 deployment weights
deployment_int8_model.npz	quantized weights/bias/scales/preprocess constants
fixed_requant_params.npz	Conv1/Conv2/FC1 M and shift, output scale
fixed_golden_result.npz	75-sample reference vs fixed output
rtl_golden_sample0.npz	layer-by-layer RTL golden sample

부록 B. 핵심 코드 요약

전체 스크립트를 그대로 복사하기보다 설계 판단과 RTL 이식에 직접 관련된 핵심 로직을 발췌한다.

B.1 100-cycle 선택

```
def find_100_cycles(rise_times):
    best_cycles = None
    best_score = np.inf
    for start in range(len(rise_times) - 100 + 1):
        candidate = rise_times[start:start+100]
        intervals = np.diff(candidate)
        steady = intervals[1:] if len(intervals) > 1 else intervals
        score = np.mean(np.abs(steady - 6.0))
        if score < best_score:
            best_score = score
            best_cycles = candidate
    return best_cycles, best_score
```

B.2 Wavelength 3648 -> 128 max-pool

```
def wavelength_max_pool(spectrum, bins=128):
    groups = np.array_split(np.arange(len(spectrum)), bins)
    return np.asarray([np.max(spectrum[idx]) for idx in groups],
                      dtype=np.float32)
```

B.3 Residual target

```
baseline = LinearRegression()
baseline.fit(wafer_train, y_train)
base_train_pred = baseline.predict(wafer_train)
residual_train = y_train - base_train_pred
# CNN/Ridge learns residual instead of absolute 40.xx um target
```

B.4 Position-preserving Tiny CNN

```
Conv2d(1,4,3,padding=1) -> ReLU -> MaxPool2d(2)
Conv2d(4,8,3,padding=1) -> ReLU -> MaxPool2d(2)
AvgPool2d(kernel_size=(5,4), stride=(5,4))
Flatten()
Linear(320,16) -> ReLU -> Dropout(0.2)
Linear(16,1)
```

B.5 Train-only normalization

```
X_train = np.log1p(X_train)
X_test = np.log1p(X_test)
x_mean = np.mean(X_train)
x_std = np.std(X_train)
X_train = (X_train - x_mean) / (x_std + 1e-8)
X_test = (X_test - x_mean) / (x_std + 1e-8)
```

B.6 INT8 symmetric quantization

```
scale = max(abs(x)) / 127
q = round_half_away_from_zero(x / scale)
q = clip(q, -127, 127).astype(np.int8)
acc_int32 = sum(q_activation * q_weight) + bias_int32
```

B.7 Fixed requantization

```
real_multiplier = input_scale * weight_scale / output_scale
M = round(real_multiplier * (1 << 24))
product = int64(accumulator) * int64(M)
q = rounded_right_shift(product, 24)
q = clip(q, -127, 127).astype(np.int8)
```

B.8 Signed round-half-away-from-zero shift

```
rounding = 1 << (shift - 1)
if value >= 0:
    result = (value + rounding) >> shift
else:
    result = -(((value) + rounding) >> shift)
```

B.9 Final PS physical reconstruction

```
baseline_um = (-0.10232342 * wafer_order) + 40.77884674  
residual_um = fc2_acc * 1.487711674e-5 + 0.00000041  
final_um = baseline_um + residual_um
```

부록 C. 주요 결과 로그 요약

```
[Full Dataset]
Generated samples : 75 / 75
Failed samples   : 0
X shape          : (75, 100, 128, 1)
y shape          : (75,)
Lots             : [1 2 3 4 5 6 7 9]

[Baselines]
Mean Baseline
  MAE : 0.2984 um
  RMSE : 0.3474 um
  R2  : -0.0197

Wafer-order Linear Baseline
  MAE : 0.1541 um
  RMSE : 0.1885 um
  R2  : 0.6998

[Direct Tiny CNN - failed]
MAE : 0.2989 um
RMSE : 0.3488 um
R2  : -0.0277

[OES Ridge]
MAE : 0.2319 um
RMSE : 0.2892 um
R2  : 0.2932

[Wafer + OES Residual Ridge]
MAE : 0.0988 um
RMSE : 0.1261 um
R2  : 0.8656
Improved lots : 7 / 8

[Residual Tiny CNN - initial successful run]
Wafer baseline MAE : 0.1541 um
CNN residual MAE   : 0.1034 um
CNN RMSE           : 0.1358 um
CNN R2             : 0.8441
Improvement        : 32.9%

[Multi-seed]
Mean MAE : 0.1163 um
MAE std  : 0.0051 um
Best MAE : 0.1121 um
Worst MAE : 0.1247 um
Mean R2   : 0.8169
Judgement : STRONG PASS

[INT8 PTQ]
FP32 MAE : 0.1247 um
INT8 MAE : 0.1232 um
INT8 R2  : 0.8056
Degradation : -0.0015 um
Judgement : STRONG PASS

[Fixed Golden]
Conv1 mismatch : 0.000547%, max diff 1
Conv2 mismatch : 0.001615%, max diff 1
FC1 mismatch   : 0
FC2 acc mismatch : 0
Final output mean/max difference : 0.0000000000 um
```

최종 판단

설계 상태 요약

지금까지의 진행은 순서와 검증 논리가 잘 잡혀 있다. 특히 random split 을 피하고 LOLO, residual 진단, multi-seed, INT8, fixed golden 을 차례로 통과한 점이 강하다. 현재부터는 모델을 더 만지는 것보다 frozen arithmetic 과 golden 을 기준으로 단순한 BRAM-based 8-MAC NPU 를 구현하는 것이 가장 합리적이다.